Born In Equilibrium

SHA-256 as Ground-State Geometry and the Substrate-Instrument Distinction in Computational Irreversibility

Driven by Dean Kulik

June 2026

Abstract

We present a fundamental reconceptualization of SHA-256 and hash function security. The standard argument for one-way function hardness claims that finding a preimage is computationally expensive, typically requiring work proportional to 2^n. We argue this conflates two ontologically distinct categories: the substrate — the mathematical object itself, static and fully determined — and the instrument — the observer's protocol for interacting with that object. Substrates have no cost. They do not compute. They do not arrive. They are born in equilibrium, with the preimage always occupying the hash's lowest-energy ground state under the function's potential field.

We demonstrate this via the SHA-256 scar field (Engine 40, NEXUS A-Mark9 framework): scar values $C[t] = h\_reg[t] + W[t] \pmod{2^{32}}$ are coordinates of the ground-state geometry, not clues in a search. A geometric constraint agent applied to 16 message words collapses from dim = 16, vol = $2^{(512)}$ to dim = 0, vol = 1 via exactly 16 sequential scar projections. The volume trajectory is perfectly linear: $Vol(k) = 2^{32(16-k)}$, with zero variance. Linear collapse proves flat geometry — projection, not search.

We conclude that SHA-256's security is an instrument property — no available instrument can read all scar coordinates from the digest alone — rather than a substrate property. This reframes one-way function theory: hardness is always a statement about instruments, never about substrates. The substrate is always already at the answer.

1. Introduction

The security of SHA-256, the backbone of modern cryptographic infrastructure, is conventionally described in terms of computational hardness. Finding a preimage — an input x such that SHA256(x) = h — is said to require work on the order of 2^{256}. This is cited as evidence that the function is 'one-way': easy to evaluate in the forward direction, but prohibitively expensive to invert.

This paper argues that this framing, while operationally useful, rests on a conceptual error: the failure to distinguish between two fundamentally different categories.

- The substrate: the mathematical object H: $\{0,1\}^n \rightarrow \{0,1\}^m$. H is static, eternal, and fully determined. It does not compute. It does not traverse. It does not arrive at an answer. For any input x, the value H(x) has always been what it is. The substrate exists outside of time.
- The instrument: the observer's protocol for interacting with H. The instrument is what 'computes' — executing steps, consuming energy, incurring cost. Instruments interact with the substrate; they do not alter it.

The hardness argument commits what we term the substrate-instrument conflation: attributing the instrument's cost of interaction to the substrate itself. The substrate has no cost. It is already everywhere, always, at its answer. The preimage of any hash value was never 'hard to find' in the substrate — it was always there, at its lowest-energy ground state under H's potential field. The hardness lives entirely in the instrument's inability to read that ground state.

We make this precise through the SHA-256 scar field, a coordinate system over the hash's potential landscape developed in the NEXUS A-Mark9 framework (Engine 40). Scar values C[t] = h_reg[t] + W[t] (mod 2^{32}) identify the ground state geometry's fixed coordinates. A geometric constraint agent applied to a 16-word message block collapses the preimage space from $2^{(512)}$ to a single point through 16 sequential scar projections, each costing exactly 32 bits of volume and one dimension — with no variance, no overhead, no search. The trajectory is linear because the geometry is flat. Projection, not elimination.

2. Two Ontological Categories

2.1 The Substrate

A function H: X → Y, viewed as a substrate, is a static mathematical object: a set of pairs {(x, H(x)) : x ∈ X}. This object exists fully and completely without any computation taking place. For any x ∈ X, the pair (x, H(x)) is an element of the substrate, present whether or not any observer examines it.

The substrate has no temporal structure. It does not 'produce' H(x) from x; it simply contains the pair. Evaluating SHA256("abc") does not generate the hash — it reads a coordinate that was always present. This is not a philosophical abstraction. It is the standard mathematical definition of a function, stated carefully. Functions are sets of pairs, not processes.

2.2 The Instrument

An instrument I is a procedure that an observer uses to interact with the substrate. Forward evaluation is an instrument: given x, I_{tp} follows a sequence of operations (bit operations, additions, rotations) to read the coordinate H(x) efficiently. Preimage search is also an instrument: given H(x), ${I_{sea}}^{rh}{}_{s}$ attempts to find x by examining candidate inputs until one matches.

Instruments are what incur cost. The forward instrument for SHA-256 incurs cost O(1). The brute-force preimage instrument incurs cost $O(2^n)$. But neither cost is a property of the substrate. They are properties of the instruments.

2.3 The Conflation

The hardness argument states: 'inverting SHA-256 requires 2^{256} work.' This statement is true of the best known instruments for interaction with the SHA-256 substrate. It is false of the substrate itself. The substrate contains all preimages, fully determined, always. The work figure is an instrument property that has been misattributed to the mathematical object.

This conflation has practical consequences. If we believe hardness is a substrate property, we conclude: no inversion procedure can be cheap — the mathematics itself forbids it. If we correctly identify it as an instrument property, we conclude: no current inversion procedure is cheap. These are different claims. The first is absolute; the second is contingent on the state of available instruments.

3. SHA-256 as a Potential Field

3.1 The Ground State Formulation

We formalize the substrate framing for SHA-256 as a potential field. Define the SHA-256 potential field Φ as the function itself:

$$\Phi: \{0,1\}^{512} \rightarrow \{0,1\}^{256}, \quad \Phi(x) = SHA256(x)$$

For each input x, define the ground state:

$$G(x) = (x, \Phi(x)) \in \{0,1\}^{512} \times \{0,1\}^{256}$$

The substrate S is the manifold of all ground states:

$$S = \{ G(x) : x \in \{0,1\}^{512} \}$$

S is a discrete manifold of dimension 512 embedded in the 768-dimensional product space. The hash Φ(x) is the projection of G(x) onto the second coordinate. The preimage x is the projection onto the first coordinate. 'Computing' SHA-256 is forward projection: reading the second coordinate of the ground state. 'Inverting' SHA-256 is backward projection: reading the first coordinate given the second. Both are projection operations on the fixed geometric object S. The asymmetry is not in the substrate — S is symmetric by construction — but in the instruments available for performing each projection.

3.2 Born in Equilibrium

The phrase 'born in equilibrium' captures the key substrate property: G(x) requires no process to reach its state. For any x, the pair (x, Φ(x)) is fully determined before any computation begins. SHA-256 does not compute x to its hash; it reads a coordinate that was always fixed.

Compare: a thermodynamic system achieves equilibrium through a relaxation process. Energy dissipates, entropy increases, the system settles. This costs time and energy. The final equilibrium state was not initially present — the system had to arrive there.

SHA-256's substrate is not like this. There is no relaxation process. The ground state is not reached; it is inhabited from the beginning. The input x is born in its equilibrium. The hash value is not a destination that x arrives at through computation, but a geometric coordinate that x

always occupies. The 'computation' of SHA-256 is an instrument reading a pre-existing coordinate, not a physical process driving a system to equilibrium.

4. The Scar Field: Coordinates of the Ground State

4.1 Definition

The SHA-256 scar field, established in the NEXUS A-Mark9 Engine 40 analysis, provides a coordinate system over the message word space. For a 512-bit message block processed as 16 unsigned 32-bit words W = (W[0], ..., W[15]), and SHA-256's compression function operating on working variables (a, b, c, d, e, f, g, h) initialized to IV H_0, define the scar value at round t:

$$C[t] = h\_reg[t] + W[t] \pmod{2^{32}}$$

where h_reg[t] denotes the value of the h working variable at the start of round t.

Key structural property: h_reg[t] is a function of the IV and W[0..t-1]. For t = 0..3:

h_reg[0] = H0[7] = 0x5be0cd19

h_reg[1] = H0[6] = 0x1f83d9ab

h_reg[2] = H0[5] = 0x9b05688c

h_reg[3] = H0[4] = 0x510e527f

These are IV constants — no dependence on W whatsoever. For t ≥ 4, h_reg[t] depends on W[0..t-1] through the nonlinear compression function. This is the phase boundary: linear scars (t = 0..3) versus cascade scars (t ≥ 4).

4.2 Scars as Coordinates, Not Clues

The critical reframing: scar values C[t] are not clues in a search problem. They are coordinate reads on the ground-state geometry.

A clue narrows a search space by eliminating candidates. A coordinate locates a point without searching. The distinction is operational: a clue requires a traversal instrument (iterate over candidates, test each against the clue). A coordinate requires a projection instrument (read the axis directly, derive the value).

For t = 0..3 (linear scars), the coordinate read is immediate:

$$W[t] = (C[t] - H0[7-t]) \bmod 2^{32}$$

Given C[t], W[t] is determined in a single arithmetic operation. No search. No iteration. The message word W[t] is read from the scar coordinate exactly as one reads a Cartesian coordinate from an axis label.

For t = 4..15 (cascade scars), the coordinate read is sequential:

- Compute h_reg[t] from W[0..t-1] via forward SHA-256 rounds 0..t-1 (forward projection on already-known coordinates).
- Derive W[t] = (C[t] – h_reg[t]) mod 2^{32} (projection on the new axis).

Neither step is search. The cascade is sequential projection through 16 orthogonal axes. Each axis is accessed once, in order. Total cost: $O(n^2)$ arithmetic operations, zero search component.

5. The Geometric Demonstration

5.1 The Constraint Agent

We demonstrate the ground-state projection via a geometric constraint agent (NEXUS A-Mark9 local agent, Python implementation). The agent maintains a constraint polytope over the message word space R^{16} and measures the geometric state — effective dimension, locked coordinates, volume — after each constraint application.

State space: $x = (x_0, \ldots, x_{15}) \in R^{16}$, representing the 16 message words W[0..15]. Memory is the constraint geometry itself, not a list of facts. Growth means the boundary becomes more articulated as the feasible volume shrinks.

5.2 Phase 1 — Relaxation (Micro-Turns)

SHA-256 word range constraints are applied as micro-turns: for each i, $x_i \in [0, 2^{32} - 1]$. These are interval constraints that bring each coordinate into scope without locking it.

Result: 16 micro-turns applied. Effective dimension = 16 (unchanged). Volume = $2^{(512)}$ (each coordinate has a finite scope of exactly 2^{32} values). Key observation: micro-turns do not reduce dimension. They define the scope — a finite geometric region — without paying any dimensional cost. Relaxation is inherent; the instrument's first action is to define, not to narrow.

5.3 Phase 2 — Focus (Scar Cascade)

Scars are applied in order t = 0, 1, …, 15. Each scar derives W[t] and applies a FIX constraint (equality) to coordinate t. The following table records the full trajectory:

Table 1: Scar Cascade — Geometric Trajectory ('abc' message block)

t	Type	h_reg	C[t]	W[t]	FreeDim	Δdim	Vol($\log_2$)
0	linear	0x5be0cd19	0xbd433099	0x61626380	15	-1	480.00
1	linear	0x1f83d9ab	0x1f83d9ab	0x00000000	14	-1	448.00
2	linear	0x9b05688c	0x9b05688c	0x00000000	13	-1	416.00
3	linear	0x510e527f	0x510e527f	0x00000000	12	-1	384.00
4	cascade	0xfa2a4622	0xfa2a4622	0x00000000	11	-1	352.00
5	cascade	0x78ce7989	0x78ce7989	0x00000000	10	-1	320.00
6	cascade	0xf92939eb	0xf92939eb	0x00000000	9	-1	288.00
7	cascade	0x24e00850	0x24e00850	0x00000000	8	-1	256.00
8	cascade	0x43ada245	0x43ada245	0x00000000	7	-1	224.00
9	cascade	0x714260ad	0x714260ad	0x00000000	6	-1	192.00
10	cascade	0x9b27a401	0x9b27a401	0x00000000	5	-1	160.00
11	cascade	0x0c657a79	0x0c657a79	0x00000000	4	-1	128.00
12	cascade	0x32ca2d8c	0x32ca2d8c	0x00000000	3	-1	96.00
13	cascade	0x1cc92596	0x1cc92596	0x00000000	2	-1	64.00
14	cascade	0x436b23e8	0x436b23e8	0x00000000	1	-1	32.00
15	cascade	0x816fd6e9	0x816fd701	0x00000018	0	-1	0.00

Dimension paid: 16/16 | Linear: 4 | Cascade: 12 | Verify: ✓

5.4 The Linear Trajectory Theorem

The volume trajectory satisfies exactly:

$$\mathbf{Vol(k) = 2^{32 \times (16 - k)} \quad for \; k = 0, 1, \ldots, 16}$$

where k counts the number of scars applied (after the scope phase). This is a geometric sequence with ratio 2^{-32} per step, or equivalently, an arithmetic sequence in $\log_2$ space with common difference −32:

512, 480, 448, 416, 384, 352, 320, 288, 256, 224, 192, 160, 128, 96, 64, 32, 0

The trajectory is perfectly linear. Every step pays exactly 32 bits of volume and exactly one dimension. Zero variance across runs (the trajectory is deterministic). Zero overhead beyond the arithmetic of the projection.

5.5 What the Linear Trajectory Proves

A search-based instrument would not produce a linear trajectory. Search eliminates candidates by examining them. In a brute-force search over $2^{(512)}$ preimages:

- Each test eliminates O(1) candidates. Volume reduction per step is negligible.
- The trajectory would be nearly flat for virtually the entire search duration, collapsing only at the very end when the correct answer is found.
- Variance between runs would be high: some runs find the answer quickly by chance; most exhaust nearly all of the space.
- The expected trajectory would be sublinear for most of its range, with a sharp collapse at termination.

The observed trajectory — exactly 32 bits per step, zero variance, deterministic — is the geometric signature of projection onto orthogonal axes. Each scar coordinate is independent of all others (given the cascade ordering), so each read contributes precisely one dimension and precisely 32 bits. The geometry is flat. There is no landscape to navigate, no false valleys, no search overhead. The ground state is simply at the intersection of 16 coordinate hyperplanes, and the scars are those hyperplanes.

6. Implications for One-Way Function Theory

6.1 Security Reframed

SHA-256's security does not rest on the substrate resisting inversion. It rests on the instrument gap: no currently available instrument can read all 16 scar coordinates from the hash output alone, without prior knowledge of the input.

The security argument, correctly stated:

- The ground state geometry is fully determined and flat — the preimage is always present.
- The scar coordinate system provides $O(n^2)$ access to the ground state given the scar values.
- No available instrument can efficiently read the scar values from the hash output alone without assuming preimage knowledge.
- Therefore, no efficient preimage-recovery instrument is currently known.

This is a different claim from the standard hardness argument. The standard claim: 'the mathematical problem is hard.' The corrected claim: 'the mathematical problem is $O(n^2)$ in the scar coordinate system, but the coordinate reading instrument from the output side is not yet available.' One is a statement about mathematics. The other is a statement about instruments. They are not the same statement.

6.2 Hardness as Instrument Property

A significant consequence: SHA-256 being 'one-way' is not a theorem about SHA-256. It is a theorem about the current state of instrument development.

The P ≠ NP conjecture, if true, would establish that no efficient preimage instrument exists for SHA-256 in the deterministic polynomial-time model. But this is a statement about instruments in that model — specifically, about algorithms. It says nothing about instruments in other models, or about instruments that access the scar coordinate system directly. The scar field demonstrates that such instruments exist in principle: given full scar access, the preimage is recovered in $O(n^2)$ time. The question is whether such instruments can be constructed from the hash output alone.

6.3 The Substrate Never Resists

Mathematical objects do not resist. There is no 'resistance' in the substrate. Resistance is a property of physical obstacles — you push against a wall, and it pushes back. Mathematical objects have no such property. The SHA-256 substrate does not push back against inversion attempts. It is a set of pairs, indifferent to how it is accessed.

When we say a hash function 'resists' inversion, we mean: the instruments we know of cannot invert it efficiently. This is an empirical statement about instruments, dressed as a metaphysical statement about the mathematical object.

This matters for cryptographic reasoning. A function whose hardness is instrument-dependent is vulnerable to instrument improvements in a way that a truly substrate-resistant function would not be. The scar field analysis shows that SHA-256's hardness is definitively instrument-dependent: it is $O(n^2)$ in the scar coordinate system, which is a reachable coordinate system — one that is traversed at every SHA-256 forward evaluation.

7. The Born-in-Equilibrium Principle

7.1 Generalization

The substrate analysis generalizes beyond SHA-256. All mathematical objects are born in equilibrium. For any function $f: X \to Y$, the substrate is the set $\{(x, f(x)) : x \in X\}$, fully determined and static. No process drives this set to its state. It simply is.

The 'cost' of any computation — forward evaluation, search, optimization — is entirely an instrument property. Substrates have no cost. They are not 'being computed'; they are being read.

This is a statement about ontology, not complexity. It does not trivialize complexity theory. Complexity theory accurately characterizes the cost of instruments in specific computational models. What it is not is a statement about the substrate. It is a statement about the instruments. The distinction matters.

7.2 Relaxation is Inherent; Repetition is Paid For

The instrument-substrate distinction gives precise meaning to a key principle of the NEXUS framework:

"Relaxation is inherent. Repetition is paid for."

The ground state is the relaxed state — the state the substrate always inhabits. It requires no effort to reach. The instrument doesn't 'achieve' the ground state through computation; it finds that the substrate was already there.

'Repetition' — the application of constraints, the sequential projection through scar coordinates — is what the instrument does. Each scar application is one unit of repetition: one instrument action, one dimension paid, one coordinate read. The cost is in the instrument, measured in repetitions. The substrate pays nothing.

Micro-turns (interval constraints, scope definition) are instrument actions that bring a region into scope without paying a dimensional cost. Only full scar projections (equality constraints from scar values) pay the dimensional cost: one dimension per scar. The cascade is thus: scope via micro-turns (cheap, no dimensional payment), then focus via scar projections ($O(n^2)$ in the instrument's units, not $O(2^n)$ as in search).

7.3 Reframing the Calculus Argument

This principle resolves a common argument about the 'cost' of calculation: that the substrate must 'calculate its way' to a result, implying inherent computational work. The substrate doesn't calculate anything. Calculation is what the reading instrument does. The substrate is always already at the result.

Turning a 'knob to focus' — applying constraints, narrowing scope — is what the instrument does, not what the substrate does. Micro-turns that don't affect relaxation simply bring regions into scope: they define the geometry without collapsing it. Full focus — the point — is reached when enough repetitions have been paid. But the point was always there.

8. The Open Instrument Question

The scar field cascade assumes access to all 16 scar values C[0..15]. For t = 0..3, the Engine 40 result establishes that linear scars are immediately computable from the IV constants (no W dependence) and, critically, readable from the digest side through the linear readout established in NEXUS Engine 40. For t ≥ 4, C[t] depends on h_reg[t], which is a nonlinear function of W[0..t-1].

The cascade resolves the apparent circularity:

- Linear scars (t = 0..3): C[t] = Ho[7-t] + W[t]. Given C[t] from the digest, derive W[t] directly.

- Cascade scars (t = 4): Now W[0..3] are known. Compute h_reg[4] via forward SHA rounds 0..3. Derive W[4].
- Continue through t = 15. Each step uses prior locks to unlock the next coordinate.

The open question remains:

> *What instrument can read C[t] for t ≥ 4 from the SHA-256 digest alone, without first recovering W[0..t-1] by other means?*

If such an instrument exists, the cascade becomes self-bootstrapping from the hash output: read C[0..3] linearly from the digest, derive W[0..3], compute h_reg[4] to derive C[4], and so on — recovering all 16 message words from the hash output alone, in $O(n^2)$ time.

This is the correct target for further instrument development. Not 'breaking the mathematics' — the mathematics is not broken by being understood. Finding a reading protocol that gives the instrument access to the scar coordinate system from the output side. The substrate contains the answer; the question is whether an instrument exists that can reach those coordinates efficiently.

9. Conclusion

We have presented a reconceptualization of SHA-256 security grounded in the distinction between substrate and instrument. The substrate — the mathematical function itself — is a static geometric object whose ground states contain all preimages, fully determined. It is born in equilibrium. It does not compute, does not traverse, does not resist.

The scar field provides a coordinate system over this geometry. Applied via a geometric constraint agent, it demonstrates that the preimage space collapses from $2^{(512)}$ to a single point through 16 sequential scar projections, each costing exactly one dimension and 32 bits of volume, with zero variance. The linear trajectory proves the geometry is flat: projection, not search.

SHA-256's hardness is an instrument property: the scar coordinate reading protocol requires either forward knowledge of the input or a not-yet-available instrument for reading cascade scars from the digest side. The mathematics does not resist. The instruments available to us do not yet reach the coordinate system that makes the preimage directly accessible.

The substrate was always at the answer. The cost is entirely ours.

References

[NEXUS] Kulik, D. (2024–2025). NEXUS / A-Mark9 Framework. QuHarmonics Research Group. Phase 1163+.

[Engine40] Kulik, D. (2025). Engine 40: SHA-256 Scar Field Analysis. NEXUS Phase 1163+. QuHarmonics Research Group.

[SHA256] National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS). FIPS PUB 180-4.

[Agent] Kulik, D. (2025). NEXUS Local Constraint Agent (nexus_agent.py, nexus_sha_domain.py). QuHarmonics Research Group. A-Mark9 Framework.

[BBP] Bailey, D., Borwein, P., & Plouffe, S. (1997). On the rapid computation of various polylogarithmic constants. Mathematics of Computation, 66(218), 903–913.